

Computer Architecture

Chapter-1

1. What, in general terms, is the distinction between computer organization and computer architecture?

Computer Organization refers to the parts of a computer system and how they work together to execute instructions and process data. Computer Organization deals with a structural relationship. This includes the design and function of the CPU, memory hierarchy, input/output systems, and their connections. It focuses on the hardware details of how a computer operates. Computer organization indicates its performance. It handles the segments of the network in a system.

Computer Architecture, on the other hand, deals with the overall design and structure of a computer system. It includes the instruction set, programming model, and features that determine the computer's capabilities and performance. It makes the computer's hardware visible. Computer Architecture deals with the functional behaviour of computer systems. It considers system performance, cost, power use, and other design choices, showing how software interacts with hardware. Architecture indicates its hardware. It coordinates the hardware and software of the system.

3. What, in general terms, is the distinction between computer structure and computer function?

Computer Function refers to what the computer does with those components. This includes executing instructions, processing data, storing information, and providing output. It's about how the computer uses its parts to perform tasks. Now we can add four main functional component of computer.

Computer Structure refers to the physical components of a computer system and their organization. This includes the CPU, memory, input/output devices, and how they are connected. It's about what parts make up the computer. Now we can add structural component of computer.

Chapter-2

1. Discuss the different generation computer.

Table 2.2 Computer Generations

Generation	Approximate Dates	Technology	Typical Speed (operations per second)
1	1946–1957	Vacuum tube	40,000
2	1958–1964	Transistor	200,000
3	1965–1971	Small and medium scale integration	1,000,000
4	1972–1977	Large scale integration	10,000,000
5	1978–1991	Very large scale integration	100,000,000
6	1991–	Ultra large scale integration	1,000,000,000

2. Discuss techniques to improve the speed of microprocessors:

1. **Clock Speed Increase:** Boosting the frequency at which the processor operates.
2. **Pipeline Architecture:** Breaking down tasks into smaller stages to execute instructions faster.
3. **Superscalar Execution:** Simultaneously executing multiple instructions with multiple execution units.
4. **Caching:** Using faster and larger caches to store frequently accessed data and instructions.
5. **Instruction-Level Parallelism:** Optimizing instruction execution order for efficiency.
6. **Branch Prediction:** Predicting and executing conditional branches ahead of time to reduce delays.
7. **Vector Processing:** Handling multiple data elements simultaneously for tasks involving large datasets.
8. **Multi-core and Multi-threading:** Using multiple cores or threads to execute tasks in parallel.
9. **Die Shrinking and Moore's Law:** Shrinking transistor size to increase processing speed and efficiency.
10. **Power Management:** Efficiently managing power consumption to optimize performance and reduce heat.

3.What is a stored program computer?

A **stored-program computer** is a type of computer where both the instructions (program) and the data are stored together in the same memory. This idea, introduced by John von Neumann, is the foundation of most modern computers.

Key Features:

1. **Unified Memory:**
 - Stores both instructions and data in the same place.
2. **Sequential Execution:**
 - Fetches and executes instructions one by one in order.
3. **Re-programmability:**
 - Can perform different tasks by loading different programs.
4. **Instruction Cycle:**
 - Fetch, decode, execute, and then move to the next instruction.

Examples:

- Early computers like **EDVAC** and **EDSAC**.
- Modern devices like PCs and smartphones.

In short, stored-program computers are versatile and powerful because they store and run programs from the same memory space.

4.List and explain the key characteristics of a computer family

1. **Compatibility:**
 - Software and accessories work with all models in the family.
2. **Scalability:**
 - Different models for different needs, from basic to powerful.
 - Easy to upgrade to a better model without changing everything.
3. **Shared Design:**
 - Similar internal parts and systems across all models.
4. **Variety:**
 - Includes desktops, laptops, and other types of computers.
 - Offers different features and sizes.
5. **Consistency:**
 - Same user experience on all models.
 - Easier to fix and support.
6. **Cost-Effective:**
 - Using shared parts saves money.
 - Software and accessories stay useful longer.

7. Backward Compatibility:

- New models can use old software and accessories.

Example: IBM System/360, where all models work well together and share common parts and software.

5. What is the key distinguishing feature of a microprocessor?

The key distinguishing feature of a microprocessor is that it integrates the functions of a computer's central processing unit (CPU) onto a single or a few integrated circuits (ICs). This compact design allows for greater efficiency, performance, and versatility in computing tasks.

Key Points:

- **Integration:** Combines arithmetic, logic, control, and storage functions in one chip.
- **Compactness:** Reduces the size and complexity of the CPU, enabling smaller and more powerful devices.
- **Versatility:** Used in a wide range of devices, from computers and smartphones to embedded systems in appliances and vehicles.
- **Efficiency:** Enhances processing speed and energy efficiency compared to earlier, larger computing systems.

This miniaturization and integration are what make microprocessors fundamental to modern electronics.

6. At the integrated circuit level, what are the three principal constituents of a computer system?

At the integrated circuit level, the three principal constituents of a computer system are:

1. Processor (CPU):

- Executes instructions and processes data using components like the ALU, CU, and registers.
- Acts as the central processing unit that controls and manages computations.

2. Memory:

- Stores data and instructions temporarily (RAM) or permanently (ROM).

- RAM provides fast access to data needed by the CPU, while ROM holds essential firmware and software.

3. Input/Output (I/O) Interfaces:

- Handles communication between the computer and external devices.
- Includes ports, buses, and controllers that manage connections to devices like keyboards, mice, monitors, printers, and storage devices.

These components together enable a computer to execute programs, store information, and interact with users and peripherals efficiently.

8.Explain Moore's law.

Moore's Law, named after Gordon Moore who observed it in 1965, describes how the number of tiny electronic switches (called transistors) on computer chips tends to double roughly every two years. This constant doubling has driven the rapid improvement in computing power we've seen over the decades, making computers faster and more affordable. However, as transistors become incredibly small, it becomes harder and more expensive to keep doubling their number, which could eventually slow down the pace of technological advancement in the future.

Chapter-3

1.Discuss the basic component of EDVAC computer

The basic components of the EDVAC computer, one of the earliest stored-program computers, include:

1.Central Processing Unit (CPU):

- Executes instructions with components like the ALU and Control Unit.

2. Memory:

- Stores data and instructions, pioneering the concept of program storage.

3. Input/Output (I/O) Devices:

- Facilitates interaction with external devices like punch cards and printers.

4. Control Panel:

- Provides manual control over operations and program loading.

5. Supporting Components:

- Includes power supply, cooling systems, clock generator, and peripheral controllers.

These components together formed the foundation for modern computer architecture, emphasizing programmability and electronic data processing capabilities.

2.What is IC and CPI?

1. IC (Integrated Circuit):

- **Definition:** An integrated circuit (IC) is a tiny electronic circuit made from semiconductor materials.
- **Function:** It performs tasks such as processing data, storing information, and controlling devices in electronic devices.

Integrated circuits (ICs) are used because they:

- **Miniaturize** electronic components,
- Are **cost-effective**,
- Enhance **reliability**,
- Improve **performance**,
- Offer **versatile** functionality

2. CPI (Clock Per Instruction):

- **Definition:** CPI, or Clock Per Instruction, measures how many clock cycles a computer needs to execute one instruction.
- **Function:** It indicates how efficiently a computer performs tasks. A lower CPI means the computer processes instructions more quickly.

3. When designing a computer which things focused?

When designing a computer, the focus is on:

- **Instruction Set Architecture (ISA):**
 - The set of instructions the computer can execute, seen by the programmer.
 - Connects hardware and software; choices affect both.
- **Implementation:**
 - **Logical Design:** High-level structure like CPU, memory, and buses.
 - **Physical Design:** Detailed circuitry and how everything is put together.
- **Organization:** The organization refers to logical aspects of an implementation. In other words it refers to the high level aspects of the design: CPU design, memory system, bus structure(s) etc.
- **Hardware:** The hardware refers to the specifics of an implementation. Detailed logic design and packaging are included here.

These strategies will help your computer designs stay competitive and effective as technology advances.

4. Design today for tomorrow's market. Which trends [HW+SW] should be followed to design a computer?

To design a computer for future markets, focus on these key trends in hardware and software:

Hardware Trends:

1. **Integrated Circuits:**
 - Chips are packing more transistors, doubling in power roughly every three years.
 - Speed is also increasing, making computers faster.
2. **RAM:**
 - Memory capacity is growing fast, becoming four times bigger every three years.
 - Access speed isn't improving as quickly, but capacities are getting bigger.

3. Disk Drives:

- Storage space is doubling every three years.
- Access times are improving, but not as rapidly.

Software Trends:

1. Memory Needs:

- Programs need more memory each year, pushing towards larger address spaces like 64-bit.

2. Compiler Technology:

- Compilers are crucial for efficiently translating code to run on hardware.
- Modern designs focus on making compilers more effective and easier to use.

6. Response Time, Throughput, Latency – definition.

- **Response Time (Latency):** This is how long it takes for a system to react to a request. It includes the time it spends thinking and the time it takes to send data back.
- **Throughput:** This tells us how much work a system can do in a certain amount of time. It shows how fast data or tasks are completed.
- **Latency:** This is the delay between starting something and seeing the first result. It depends on how fast information travels and how quickly it's processed. Low latency is really important for things that need quick responses, like video calls or online games.

Chapter-4

1. Discuss the different types of CPU component.

• Arithmetic Logic Unit (ALU):

- Performs arithmetic (addition, subtraction) and logic (AND, OR, NOT) operations crucial for computation.

- **Control Unit (CU):**

- Directs the flow of data and instructions within the CPU, ensuring tasks are executed in the correct sequence.

- **Registers:**

- Provide fast storage for data, addresses, and intermediate results, essential for efficient processing.

- **Cache Memory:**

- Stores frequently accessed data to reduce the time needed to fetch information from main memory.

- **Buses:**

- Facilitate communication between the CPU and other components, ensuring data transfer and control signals are managed effectively.

Chapter-5

1. Difference between RISC and CISC Processor.

Difference between RISC and CISC Processor

S.N o.	RISC	CISC
1.	RISC is a reduced instruction set.	CISC is a complex instruction set.
2.	The number of instructions is less as compared to CISC.	The number of instructions is more as compared to RISC.
3.	The addressing modes are less.	The addressing modes are more.
4.	It works in a fixed instruction format.	It works in a variable instruction format.
5.	The RISC consumes low power.	The CISC consumes high power.
6.	The RISC processors are highly pipelined.	The CISC processors are less pipelined.
7.	It optimizes the performance by focusing on software.	It optimizes the performance by focusing on hardware.
8.	Requires more RAM.	Requires less RAM.

RISC does not support an array

CISC supports array.

2.How many addressing modes are there?

Based on the number of addresses used for instructions, machines are categorized as:

- **3-address machines:** Instructions can operate on and reference three addresses or operands.
- **2-address machines:** Instructions work with and reference two addresses or operands.
- **1-address machines:** Instructions handle and reference one address or operand.
- **0-address machines:** Instructions implicitly reference operands from the top of a stack or use a fixed location in memory.

These categories describe how instructions are structured and interact with memory or registers in a computer architecture.

3. Register-Register, Memory-Memory, Register-Memory, Memory-Register Advantages and Disadvantages.

Register-Register Machines

Advantages:

- Simple and fixed-length instructions
- Easy for compilers to generate code
- Consistent execution time for instructions
- Less memory traffic

Disadvantages:

- More instructions needed
- Wastes space for simple instructions

Register-Memory Machines

Advantages:

- More compact instructions
- Fewer loads needed if one operand is already in memory

Disadvantages:

- One operand is destroyed in operations, complicating compiler code generation

Memory-Memory Machines

Advantages:

- Compact code
- Easy code generation

Disadvantages:

- Variable instruction lengths
- Variable execution times
- High memory traffic

Chapter-6

Addressing mode: An addressing mode in computing determines how a CPU accesses data in memory. It defines rules for calculating where data is stored or retrieved during program execution. Common modes include immediate (data is in the instruction), direct (address is in the instruction), indirect (address is in a register or memory), and others like register, indexed, relative, and stack modes. These modes help CPUs efficiently manage data access and storage based on different programming needs.

Figure: The image shows how data travels between memory and the CPU. It's like a highway system with lanes (data bus) for moving information.

The interesting part, addressing mode, isn't there. Think of addressing mode like giving directions - it tells the CPU how to find the specific house (data location) on the memory street. The image shows the road but not how to get to a specific address.

,

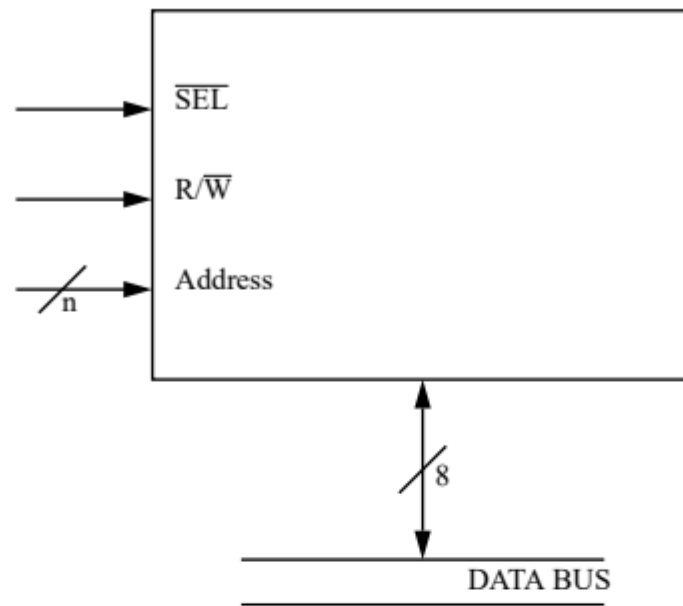


FIGURE 4.1 A byte-organized memory

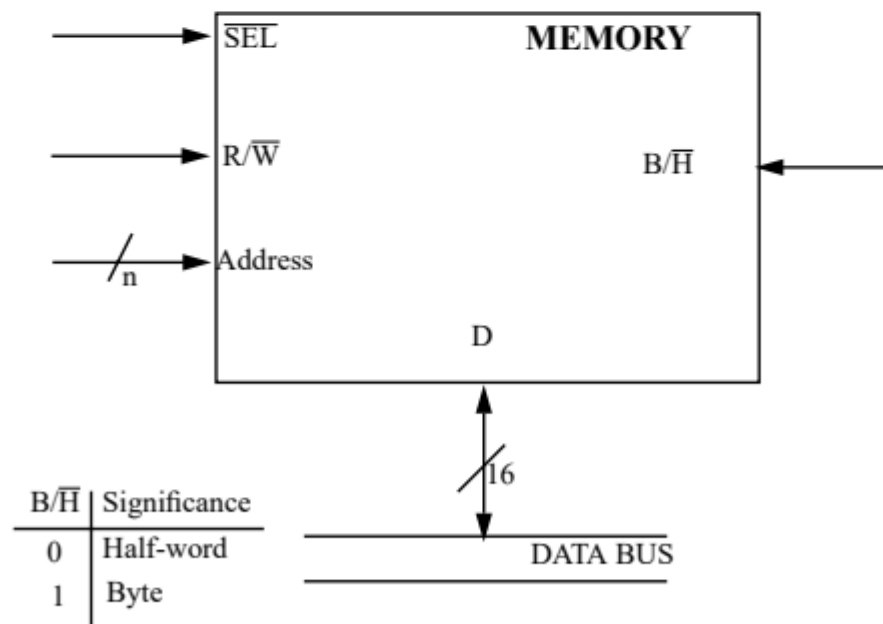


FIGURE 4.2 A memory that can be byte and half-word accessed

Chapter-7

1.How can we evaluate/find out branch condition? How many ways? [Condition Code, Condition Register, Compare and Branch], explain these.

To evaluate branch conditions, there are three main ways:

- **Condition Code (CC):**
 1. The ALU sets special bits in a flags register (like zero result, carry, or overflow) automatically during operations.
 2. The advantage is no extra work is needed to set these codes.
 3. The disadvantage is that they restrict instruction reordering, complicating assembly programming and compiler optimizations.
- **Condition Register (CR):**
 1. A specific register holds the result of a comparison.
 2. The advantage is it allows for simple implementation and flexible instruction ordering.
 3. The disadvantage is it requires a clear instruction to set the register.
- **Compare and Branch:**
 1. This method combines the comparison and branch operations into a single instruction.
 2. The advantage is it simplifies the process by reducing the number of instructions.
 3. The disadvantage is it can require complex encoding and may involve more work in a single clock cycle.

2.Discuss CPU Load and Store instructions! Memory read/write.

Load/Store Instructions

- **Address Calculation (ALU) / Memory reference:** The CPU uses the ALU to add a base address from register A to an offset. This gives the memory address.

The calculated address is stored in the Memory Address Register (MAR).

- $ALU_{output} = A + offset$
- $MAR = ALU_{output}$

- **Memory Read/Write:**

- **Store Instruction:** Data in the Memory Data Register (MDR) is written to the address in MAR.
 - $M[MAR] = MDR$ (memory at MAR is written with MDR content)
 -
- **Load Instruction:** Data from the address in MAR is read into the MDR.
 - $MDR = M[MAR]$ (MDR is loaded with memory content at MAR)

- **Write Back:**

Data from the MDR is written to the destination register specified in the instruction.

- $register[d] = MDR$

Chapter-8

1. Which types of actions are taken by control unit to manage interrupts?

When an interrupt occurs, the Control Unit takes the following actions to manage it:

1. **Save the Current PC:** The Control Unit saves the current Program Counter (PC), which holds the address of the interrupted instruction, into the Interrupt Address Register (IAR), also known as the Exception Program Counter.
2. **Call the Interrupt Handler:** The Control Unit transfers control to the operating system by going to a specific address to call an interrupt handler.

2. How to track interrupts.

There are two simple ways to track the reason for an interrupt:

1. **Status Register:** Each interrupt writes its code into a special register before the operating system takes over. This code tells the system why the interrupt happened.

2. **Vectored Interrupts:** In this approach every kind of interrupt forces a call to a different address, the address where its interrupt is located.

3. Classification of Interrupts.

1. **User Request vs. System Interrupts:**
 - **User Requests:** These are interrupts started by the program itself.
 - **System Interrupts:** These are triggered by the system or hardware.
2. **Maskable vs. Nonmaskable Interrupts:**
 - **Maskable Interrupts:** Can be controlled by the program, like handling math errors or breakpoints.
 - **Nonmaskable Interrupts:** Always need the CPU's attention for critical issues, like hardware failures.
3. **Within vs. Between Instructions:**
 - **Within Instructions:** Immediate interrupts like power failures or memory errors stop the program right away.
 - **Between Instructions:** Interrupts like I/O requests are handled during breaks in the program.
4. **Synchronous vs. Asynchronous Interrupts:**
 - **Synchronous Interrupts:** Happen at specific points in the program, like math errors or breakpoints.
 - **Asynchronous Interrupts:** Happen unexpectedly, like hardware failures or I/O requests.
5. **Resume vs. Terminate:**
 - **Resume:** The program continues after handling the interrupt and goes back to normal.
 - **Terminate:** The program stops completely due to the interrupt, ending all further processing.

Chapter-9

1. What is memory Hierarchy? Discuss.

In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behaviour.

Levels: Organized in multiple levels, each larger and slower than the previous.

1. **Registers:**
 - **Very Fast:** Inside the CPU, the quickest type of memory.
 - **Tiny:** Can only hold a small amount of data, used for immediate calculations.
2. **Cache:**
 - **Fast:** Located close to the CPU, faster than main memory.
 - **Small:** Stores data we use a lot to speed up access.
3. **Main Memory (RAM):**
 - **Quick:** Where our computer keeps active programs and data.
 - **Moderate Size:** Can hold the operating system and open applications.
4. **Secondary Storage:**
 - **Slower:** Includes hard drives and SSDs, slower than RAM.
 - **Large Capacity:** Stores your files, programs, and data long-term.
5. **Tertiary Storage:**
 - **Slowest:** Used for backups, like external drives and CDs.
 - **Very Large:** Good for storing lots of data that we don't need to access often.

2. Explain the principle of locality memory hierarchy.

The principle of locality means programs frequently access specific and nearby memory addresses. There are two types:

1. **Temporal Locality:** Recently used addresses will probably be used again soon.
2. **Spatial Locality:** Addresses close to a recently used address will probably be used soon.

According to the 90/10 rule, a program spends 90% of its time using only 10% of its memory addresses. To optimize, memory is organized in a hierarchy:

- **Top Levels:** Smallest, fastest, and closest to the CPU (like caches).
- **Lower Levels:** Larger, slower, and cheaper (like RAM and hard drives).

2. Define hit rate, miss rate, hit penalty ,miss penalty.

- **Hit Rate:**

- **Definition:** The percentage of memory accesses that are found in the higher (faster) levels of the memory hierarchy is called the "hit rate."
- **Example:** If 90 out of 100 memory accesses are found in the cache, the hit rate is 90%.

- **Miss Rate:**

- **Definition:** The percentage of memory accesses that are found in the higher (faster) levels of the memory hierarchy fetch it from slower memory levels instead.
- **Example:** If 10 out of 100 memory accesses are not found in the cache, the miss rate is 10%.

- **Hit Penalty:**

- **Definition:** The time required to access data from the higher (faster) level of memory after a hit occurs.
- **Example:** If accessing data from the cache takes 5 nanoseconds, the hit penalty is 5 nanoseconds.

- **Miss Penalty:**

- **Definition:** The time required to access data from the lower (slower) level of memory after a miss occurs.
- **Example:** If accessing data from the main memory (after a cache miss) takes 100 nanoseconds, the miss penalty is 100 nanoseconds

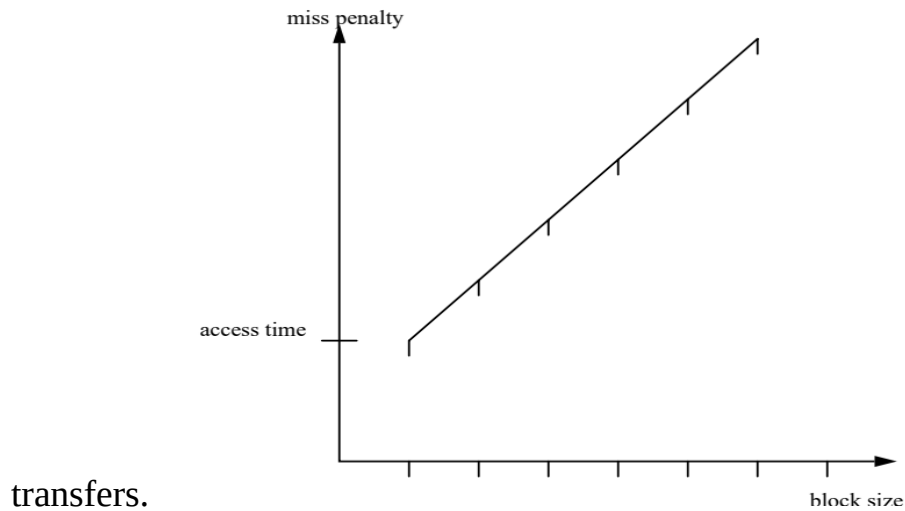
3. Discuss the block size relationship between miss penalty and miss rate.

Block Size, Miss Penalty, and Miss Rate Relationship

Miss Penalty and Block Size:

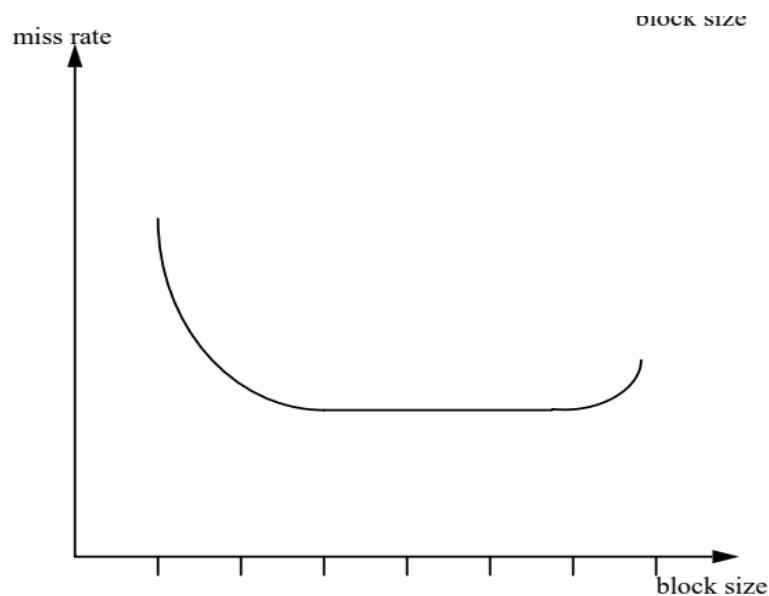
- **Definition:** Miss penalty is the time to fetch data after a cache miss.

- **Relation:** Initially rises with block size due to more data transferred. likely-to-be-used data. However, excessive block size can lead to wasted



Miss Rate and Block Size:

- **Relation:** Falls initially with block size as more nearby data is fetched. This benefits spatial locality. Beyond a certain size, miss rate rises because larger blocks may fetch more data than needed, replacing useful data too soon.



Chapter-10

1. Cache Memory Details.

Cache memory is a small, fast storage area near the CPU that helps speed up data access. It temporarily stores data and instructions. Cache memory Boosts CPU performance. It enhances system speed and efficiency

Levels

1. **L1 Cache:** Very small (16KB to 64KB), ultra-fast cache directly on each CPU core.
2. **L2 Cache:** Larger than L1 (256KB to several MB), located near the CPU cores but slightly slower, serving as a secondary cache for storing recently used data.
3. **L3 Cache:** Largest (a few MB to tens of MB), shared among all CPU cores, slower than L1 and L2, but faster than main memory (RAM), optimizing performance by providing a common pool of cached data.

2. Write down the block placing techniques in cache.

Block Placing Techniques in Cache

1. **Directly Mapped Cache:**
 - Each block has one specific place it can go in the cache.
 - Simple and easy to manage.
2. **Fully Associative Cache:**
 - A block can be placed anywhere in the cache.
 - Very flexible but more complex to manage.
3. **Set Associative Cache:**
 - A block can go in a specific set of places.
 - The cache is divided into groups called sets.
 - Within each set, a block can be placed in any spot.
 - If a set has n spots, it's called n-way set associative cache.

3.Problems of Direct Map and Fully Associative Mapping. How to solve using other techniques?

Problems of Direct Mapped and Fully Associative Mapping

1. Direct Mapped Cache:

- **Problem:** Each block has only one specific place in the cache. This can lead to frequent conflicts if multiple blocks map to the same cache location, causing frequent cache misses.

2. Fully Associative Cache:

- **Problem:** A block can be placed anywhere in the cache, making it flexible but complex. Finding and replacing blocks can be slow and resource-intensive.

Solution: Set Associative Cache

- **Balances Flexibility and Simplicity:** Divides cache into sets with multiple spots.
- **Reduces Conflicts:** Blocks have multiple placement options within a set.
- **Easier Management:** Search and replacement are quicker and simpler than in fully associative caches.

4.Describe the replacing algorithms of cache memory. [Three techniques].

Cache Replacement Algorithms

1. Random Replacement:

- **Description:** Picks a block randomly from the cache to replace.
- **Advantage:** Simple to do, but doesn't think about which blocks are used most.
- **Disadvantage:** Might replace blocks that were just used, which could slow things down.

2. Least Recently Used (LRU):

- **Description:** Keeps track of when each block was last used and replaces the one that hasn't been used for the longest time.
- **Advantage:** Tries to keep frequently used data in the cache.
- **Disadvantage:** Needs extra hardware to track usage, which can make it more complex and costly.

3. First-In-First-Out (FIFO):

- **Description:** Replaces the oldest block in the cache, no matter how often it's used.
- **Advantage:** Simple and easy to understand.

- **Disadvantage:** Doesn't consider which blocks are used most, so it might replace useful data.

These methods help decide which block gets replaced when new data needs space in the cache, aiming to balance simplicity and efficiency based on how often data is accessed.

EXTRA

1. Why memory hierarchy needed?

The memory hierarchy is necessary because it balances between speed, cost, and capacity in computer systems:

- **Speed:** Higher levels like registers and cache are faster and closer to the CPU, allowing for quicker access to frequently used data.
- **Cost:** Faster memory technologies are more expensive per unit of storage. By using slower but cheaper memory at lower levels, overall costs are reduced.
- **Capacity:** Lower levels like disk storage offer larger capacity but slower access times, suitable for storing vast amounts of data.

This hierarchy ensures that computers can efficiently manage data storage and access, optimizing both performance and cost-effectiveness.

3. How the hierarchy organized?

The memory hierarchy is organized in layers from fastest to slowest and smallest to largest capacity:

- **Registers:** Located directly in the CPU, they are the fastest but smallest in capacity.
- **Cache:** Close to the CPU, it stores frequently accessed data for quick retrieval.
- **Main Memory (RAM):** Slower than cache but larger in capacity, used for active programs and data.
- **Storage (Hard Drives, SSDs):** Provides the largest capacity but slower access times, used for long-term data storage.

This organization ensures that computers can access data quickly when needed, balancing speed and storage capacity based on the task at hand.

3.What are the methods of accessing methods of computer memory?

There are two main methods for accessing computer memory:

- **Sequential Access:** Accesses data in a linear, sequential order. It reads or writes data one after another, like reading a book from start to finish.
- **Random Access:** Allows direct access to any memory location, without needing to access preceding locations first. It's like jumping to any page in a book without reading through all previous pages.

These methods determine how quickly and efficiently data can be retrieved or stored in computer memory, depending on the type of storage used.

4.What is CPU register?

A CPU register is a tiny storage space inside the computer's processor. It holds data and instructions that the processor is actively using to perform tasks. Registers are incredibly fast, making operations quicker than fetching data from memory. They help the CPU work efficiently by keeping essential information close at hand.

5.Discuss the role of Control and Status register in CPU.

Control and status registers in a CPU handle important tasks:

- **Control Registers:** Manage how instructions execute and control CPU functions like memory access and interrupt handling. They store settings that dictate how operations are performed, such as enabling or disabling interrupts, managing memory protection, and switching between different operating modes.
- **Status Registers:** Hold information about the CPU's current state, such as whether it's busy processing data, if an operation is complete, or if an error has occurred. These registers contain flags and indicators that reflect various conditions within the CPU, like arithmetic overflow, carry, or zero results.

These registers help the CPU run programs smoothly by overseeing operations and reporting on what's happening inside the processor.